Log Ingestion and Query Service

Overview

Build a service that ingests high volumes of structured logs, stores them efficiently, and allows users to query and aggregate them.

Think of it as a simplified version of Datadog or Grafana Loki: applications send logs to your API, and your service makes those logs searchable and analyzable.

Expected timeline: 1–2 weeks

What You Are Building

Your service must address three main concerns:

1. **Ingestion**
   An API that accepts individual or batched structured log entries, validates them, and stores them efficiently.
2. **Querying**
   An API that supports filtering logs by service, level, time range, attributes, and message content, as well as aggregating logs into time buckets and grouping them by supported dimensions.
3. **Retention**
   Logs should not be stored indefinitely. Provide a configurable retention policy for deleting expired data.

A log entry must contain:

- A timestam
- A level: `debug`, `info`, `warn`, or `error`
- A service name
- A message
- A collection of arbitrary key/value attributes, such as `user_id`, `request_id`, or `region`

How you store and query the attribute collection is one of the most important design decisions in this project.

This service will be tested under load. We will run a load generator against it and verify that ingestion remains reliable and queries remain fast while the system contains more than one million rows.

A solution that is correct but cannot meet the performance requirements is not considered complete.

Core Requirements

- Implement the required API contract exactly as described below.
- Support per-entry validation for ingestion batches.
- Support freely combinable query filters.
- Support time-bucketed aggregation.
- Support cursor-based pagination.
- The complete system must start with:

  docker compose up

- The service must include a README covering:
  - Setup and usage
  - API documentation
  - Schema and index design
  - Attribute storage strategy
  - Retention strategy
  - Measured performance results
  - Known limitations

Resource Limits

The solution will run with the following container limits:

- **Application container:** 0.5 CPU and 256 MB RAM
- **PostgreSQL container:** 1 CPU and 1 GB RAM

Additional infrastructure may be used, provided that PostgreSQL remains the source of truth for both reads and writes.

Required API Contract

We will run the same automated load generator against every submission. It expects the exact endpoints, paths, and response structures described below.

You may add additional endpoints, but the required endpoints must exist exactly as specified.

If the load generator cannot communicate with your service, the submission cannot be graded.

The service must:

- Listen on port `8080` inside the application container
- Be exposed as `localhost:8080` in `docker-compose.yml`

`GET /health`

Returns HTTP `200` with any response body once the service is ready to accept traffic.

The service should only report itself as healthy after:

- The database connection has been established
- Database migrations have been applied
- The service is ready to accept logs

The load generator will poll this endpoint before starting.

`POST /logs` — Ingest Logs

This endpoint always accepts a batch. A batch containing one log entry is valid.

Request


```
{
  "logs": [
    {
      "timestamp": "2026-07-20T14:32:01.123Z",
```

```
    "level": "error",
    "service": "checkout",
    "message": "payment declined",
    "attributes": {
      "user_id": "42",
      "region": "eu-west",
      "retries": 3
    }
  }
 ]
}
```

Validation Rules

Each log entry must satisfy the following rules:

- `timestamp`
  - Required
  - Must be a valid ISO 8601 timestamp
  - Must not be more than five minutes in the future
- `level`
  - Required
  - Must be one of:
    - `debug`
    - `info`
    - `warn`
    - `error`
- `service`
  - Required

    - Must be a non-empty string
- `message`
    - Required
    - Must be a non-empty string
- `attributes`
    - Optional
    - Must be a flat object
    - Values may be strings, numbers, or booleans
    - Nested objects and arrays are not allowed
- Nested objects and arrays are not allowed

Batch Behavior

An invalid entry must not cause the entire batch to fail.

The service must:

- Accept valid entries
- Reject invalid entries
- Return the array index and rejection reason for each invalid entry

Response

Return HTTP `200` when at least one entry is accepted.

Return HTTP `400` when:

- All entries are rejected
- The request body contains malformed JSON
- The request does not match the expected top-level structure

Example response:


```
{
  "accepted": 9,
  "rejected": [
    {
      "index": 3,
```

```
    "reason": "invalid level: 'critical'"
  }
 ]
}
```

`GET /logs` — Query Logs

All query parameters are optional and may be freely combined.

Parameter	Meaning	Example
`service`	Exact service-name match	`service=checkout`
`level`	Exact level match	`level=error`
`since`	Inclusive start of the time range	`since=2026-07-20T14:00:00Z`
`until`	Exclusive end of the time range	`until=2026-07-20T15:00:00Z`
`attr.<key>`	Attribute equality, compared as strings	`attr.user_id=42`
`q`	Case-insensitive substring match on `message`	`q=declined`
`limit`	Maximum number of results; default `100`, maximum `1000`	`limit=500`
`cursor`	Opaque cursor returned by a previous response	`cursor=eyJpZCI6...`

Sorting

Results must be sorted by timestamp in descending order.

The ordering must remain deterministic when multiple logs have the same timestamp.

Response

```
{
  "logs": [
    {
      "id": "any-unique-id",
      "timestamp": "2026-07-20T14:32:01.123Z",
      "level": "error",
      "service": "checkout",
      "message": "payment declined",
      "attributes": {
        "user_id": "42"
      }
    }
  ],
  "next_cursor": "eyJpZCI6..."
}
```

`next_cursor` must be `null` when no additional results are available.

The cursor format is implementation-defined. The load generator will treat it as an opaque value and pass it back unchanged.

Invalid Parameters

Return HTTP `400` with the following structure when query parameters are invalid:

```
{
  "error": "<description>"
}
```

Examples of invalid input include:

- Invalid timestamps
- `until` earlier than `since`
- Unsupported log levels
- Non-numeric limits
- Limits outside the supported range
- Invalid or malformed cursors

`GET /logs/aggregate` — Aggregate Logs

This endpoint returns time-bucketed log counts.

It supports the same filters as `GET /logs`:

- `service`
- `level`
- `attr.<key>`
- `q`

It also accepts the following aggregation parameters:

Parameter	Required	Meaning	Example
`since`	Yes	Inclusive start of the aggregation range	`since=2026-07-20T14:00:00Z`
`until`	Yes	Exclusive end of the aggregation range	`until=2026-07-20T15:00:00Z`
`bucket`	Yes	Bucket size: `1m`, `5m`, `1h`, or `1d`	`bucket=1m`
`group_by`	No	Group results by `service` or `level`	`group_by=service`

Response

Return one row for each bucket and group combination.

Results must be ordered by bucket start time in ascending order.

Empty buckets may be omitted.

When `group_by` is not provided, `group` must be `null`.

```
{
  "buckets": [
    {
      "start": "2026-07-20T14:00:00Z",
      "group": "checkout",
      "count": 118
    },
    {
      "start": "2026-07-20T14:00:00Z",
```

```
      "group": "auth",
      "count": 42
    },
    {
      "start": "2026-07-20T14:01:00Z",
      "group": "checkout",
      "count": 97
    }
  ]
}
```

Invalid parameters must return HTTP `400` using the same error format as `GET /logs`.

Everything beyond this API contract—including retention configuration, administrative APIs, dashboards, and internal architecture—is left to your design.

Optional Features and the Load Generator Contract

We run one load generator against every submission. It is written once and is not customised per candidate. Anything you add on top of the core service must therefore keep that single load generator working without any per-project configuration.

The Golden Rule

Extras are additive, never subtractive.

An optional feature may add endpoints, headers, response fields, or configuration. It must never:

Remove or rename a required endpoint

Change the shape or types of a required response

Introduce a new required request parameter or header on a required endpoint

Cause a request that would have succeeded on the core service to fail

If a feature cannot satisfy this, it must be disabled by default.

Default Posture: Zero Configuration

A plain `docker compose up` with no environment file, no arguments, and no manual setup must produce a service that:

Serves `GET /health`, `POST /logs`, `GET /logs`, and `GET /logs/aggregate` exactly as specified

Accepts unauthenticated requests on all four

Applies no rate limit, quota, or tenancy restriction that the load generator could hit

This is the configuration we grade performance against. If we have to read your README to get a request through, the submission is treated as failing the contract.

Authentication and API Keys

If you implement authentication, API keys, or multi-tenancy, follow this contract exactly.

Configuration

Variable	Default	Meaning
`AUTH_ENABLED`	`false`	Master switch for all authentication and authorization
`LOADGEN_API_KEY`	unset	Key seeded at startup with full ingest + query permissions

Rules:

`AUTH_ENABLED` must default to `false`. With it unset or false, the service behaves exactly as the unauthenticated core service.

When `AUTH_ENABLED=true` and `LOADGEN_API_KEY` is set, the service must idempotently seed that key at startup, before reporting healthy, with permission to ingest and query all data. Restarting the service must not invalidate it.

Seeding must be part of startup or migration. No admin call, SQL snippet, or manual step may be required.

If `AUTH_ENABLED=true` and `LOADGEN_API_KEY` is unset, the service must still start and remain healthy; it simply has no seeded key.

Credential Transport

Primary: `Authorization: Bearer <key>`

You may additionally accept `X-API-Key: <key>`, but `Authorization: Bearer` must always work.

Credentials never go in the query string or request body.

Status Codes

Condition	Status	Body
Missing or malformed credential	`401`	`{"error": "<description>"}`
Valid credential, insufficient scope	`403`	`{"error": "<description>"}`
Rate limit or quota exceeded	`429`	`{"error": "<description>"}` plus `Retry-After`

Authentication failures must never return `500`, and must never return `200` with an empty result set.

Exemptions

`GET /health` is always unauthenticated, regardless of `AUTH_ENABLED`. The load generator polls it before it has any credentials.

The load generator is not told in advance whether your build has auth. It always sends `Authorization: Bearer <key>` on the three data endpoints. When `AUTH_ENABLED=false`, an unrecognised `Authorization` header must be ignored, not rejected.

Multi-Tenancy

If logs are scoped per tenant, the seeded load generator key must resolve to exactly one tenant, and all four required endpoints must operate within it transparently. Tenant identity is never a required request parameter — it is derived from the credential. Response shapes do not change.

Rate Limiting and Backpressure

Rate limiting must be off by default, or must exempt the seeded load generator key.

Backpressure is legitimate engineering: shedding load with `429` or `503` plus `Retry-After` is better than crashing. Understand, however, that the load generator counts shed requests as not ingested. They do not contribute to your throughput number.

Never respond `200` to a batch you have not durably accepted.

CI Requirement

Your pipeline must run the required-contract smoke test in both configurations:

`AUTH_ENABLED=false` — all four endpoints reachable with no credentials

`AUTH_ENABLED=true` with `LOADGEN_API_KEY` set — all four endpoints reachable with the seeded bearer token, and rejected with `401` without it

If you implement no optional features, only the first configuration applies.

README Requirement

List every optional feature you implemented, its default state, the environment variables that control it, and confirmation that `docker compose up` with no configuration yields the plain core service.

Performance Targets

We will test the system using our own load generator.

The solution must meet the following baseline targets:

- Sustain at least **15,000 logs per second**
- Avoid dropped requests and application crashes during sustained ingestion
- Return the primary aggregation query in under **1 second at p95**
- Maintain query performance while ingestion is active

- Handle approximately **1,000,000 stored log records**
- Assume those records represent approximately one month of data
- Make newly ingested data queryable within **20 seconds**
- Support one aggregation request per second during the ingestion test

The environment will be limited to:

- PostgreSQL: 1 CPU and 1 GB RAM
- Application: 0.5 CPU and 256 MB RAM

Higher ingestion throughput may earn additional credit. For example:

- 20,000 logs per second
- 25,000 logs per second
- Higher sustained rates

Run your own load tests before submitting.

Include the following in the README:

- Test environment
- Dataset size
- Batch size
- Ingestion rate
- Query rate
- Query latency percentiles
- Resource usage
- Bottlenecks discovered
- Optimizations applied

We want evidence that you measured the system rather than relying on assumptions.

What We Are Evaluating

This project is intentionally underspecified. How you fill in the gaps is part of the evaluation.

Area	What We Are Evaluating
Architecture	Schema design, attribute storage strategy, data flow, and project structure
Performance	Indexes aligned with query patterns, ingestion throughput, query latency, and behavior under concurrent load

Retention	Expired-data deletion without long-running locks, excessive table bloat, or major ingestion disruption
Reliability	Validation, error handling, malformed input handling, empty ranges, invalid cursors, and other edge cases
Code quality	Readable TypeScript, strong typing, clear abstractions, and maintainable structure
Security	Parameterized queries and safe dynamic-query construction; SQL injection is disqualifying
Separation of concerns	Query-building and persistence logic separated from HTTP handlers
Infrastructure	A Docker Compose setup that works on the first run and applies migrations automatically
CI	A meaningful pipeline that builds, tests, and validates the project
Documentation	Clear setup instructions, API documentation, design reasoning, measured results, and acknowledged limitations
Creativity and polish	Useful improvements beyond the minimum requirements

Stretch Goals

Stretch goals are optional. Prioritize a reliable and performant core implementation over incomplete extras.

Possible additions include:

- A dashboard for viewing and filtering logs
- Operational metrics for ingestion and query performance
- Alerting rules that trigger a webhook when an error threshold is exceeded
- A live-tail endpoint
- Pre-aggregated rollup tables
- A custom query language
- Multi-tenancy using API keys
- Data compression
- Rate limiting
- Dead-letter handling
- Backpressure support
- Additional observability

You may also propose and implement your own enhancement.

Deliverables

1. **GitHub repository**
    - Clean and readable commit history
    - Incremental progress should be visible
2. **Working Docker Compose setup**
    - The complete solution starts with `docker compose up`
3. **Passing CI pipeline**
    - The pipeline should perform meaningful build, test, and validation steps
4. **README**
    - Setup instructions
    - API documentation
    - Schema design
    - Index design
    - Attribute storage strategy
    - Retention strategy
    - Load-test methodology
    - Measured performance results
    - Known limitations
5. **Demo**
    - Be prepared to walk through the project
    - Explain the architecture and major trade-offs
    - Justify the schema and indexes
    - Run `EXPLAIN` or `EXPLAIN ANALYZE` on important queries
    - Trace ingestion and query code paths
    - Debug or extend a feature live

A Note on AI Usage

You are welcome and expected to use AI tools. We use them in our daily work as well.

What matters is that you understand the system you submit.

During the demo, you may be asked to:

- Explain the schema
- Justify the indexes
- Walk through important code paths
- Explain performance trade-offs
- Diagnose a problem
- Modify or extend the implementation live

Code that you cannot explain does not count as completed work.